

HAVEN

Rapport Technique Officiel — V1 Majeure

Chaîne de production visible (puits → seau → stockage → citerne), goulots d'étranglement compréhensibles à l'œil, méta-progression (Perles, Renaissance, atouts permanents). Dépôt : **scuillerjack-oss/haven-v0** · Branche : **main**.

Convention de ce rapport : **FAIT** = codé, testé et déployé sur l'URL réelle. **PRÉVU** = décidé par le cahier des charges mais non construit dans cette V1, faute de temps ou par choix de ne pas surconstruire. Aucune fonctionnalité n'est déclarée terminée seulement parce que le code existe.

0. État initial réellement trouvé et repères Git

Avant cette mission, le dépôt contenait la V0 telle que livrée et déployée : un idle abstrait (jardin à 7 paliers visuels, 4 productions génériques). Aucune trace d'une V1 antérieure dans le code du dépôt — les documents « HAVEN_V1_Mission_Claude.pdf » et l'étude Idle référencés dans le cahier des charges directeur sont des documents de cadrage, pas du code déjà produit ici. Vérifié par git log et l'historique des workflows GitHub Actions avant toute modification, comme demandé.

- Dernier commit avant cette mission : 567fa24 — « Ajoute le rapport technique officiel V0 ».
- Premier commit de cette mission : e1ee9be — correctifs mobile/responsive (jalon 1).
- Dernier commit de cette mission : b36df5e — complète la matrice de tests obligatoire.
- 4 commits de fond entre les deux, un par jalon stabilisé, comme demandé en section 32.

1. Diagnostic des bugs mobile / responsive (priorité zéro)

1.1 Interactions mortes après sortie/retour

Cause identifiée par audit de code (non reproductible littéralement en environnement headless sans un vrai iPhone, mais cohérente avec un défaut connu documenté de WebKit) : la reprise ne s'appuyait que sur l'événement `visibilitychange`. En PWA « standalone » sur iOS Safari — exactement le mode d'installation recommandé dans la procédure bêta de la V0 — cet événement est documenté comme peu fiable à lui seul. S'il ne se déclenchait pas, `lastTickTime` et `state.lastSeen` restaient bloqués sur un instant d'avant la mise en arrière-plan. Aggravant : si la moindre exception survenait pendant le traitement de la reprise (calcul hors-ligne, rendu de la modale de retour), la fonction s'interrompait avant de remettre l'horloge à jour, et la boucle continuait indéfiniment de calculer ses deltas depuis un passé de plus en plus obsolète.

Correctif : redondance de signaux (`visibilitychange` + `pagehide/pageshow` + `focus/blur`), un bloc `try/finally` qui garantit la remise à l'heure de la boucle quoi qu'il arrive, une garde anti-doublon (250 ms) contre des signaux redondants pour un seul événement réel, et une garde anti-duplication de l'intervalle principal (`restartLoop()`).

1.2 Largeur/hauteur initiale incorrecte avant recalage

Cause : `height: 100dvh` se recalcule de façon asynchrone quand la barre d'adresse mobile se rétracte après le chargement — la scène SVG, en enfant flex de cette hauteur instable, héritait de cette taille incorrecte puis « sautait » une fois la valeur stabilisée. Remplacé par `height: 100vh` puis `100svh` (petit viewport, garanti stable dès le premier paint, amélioration progressive).

1.3 Validation

`scripts/check-mobile-resume.mjs` (Playwright, câblé dans la CI) : rejoue un cycle `suspend/resume` complet et vérifie que le tiroir réagit toujours au clic ensuite ; vérifie l'absence de débordement horizontal et la pleine largeur à 5 largeurs portrait représentatives (320 à 480 px) ; vérifie l'idempotence du crédit hors-ligne face à des signaux de reprise redondants. Les trois passent en CI sur un vrai runner GitHub Actions, pas seulement en local.

Limite honnête : ce diagnostic est un audit de code rigoureux, pas une confirmation sur un appareil iOS réel. La bêta suivante doit confirmer sur le téléphone exact où le bug a été observé.

2. Architecture finale

JavaScript vanilla (ES modules), Vite, sans framework UI, comme en V0. Le moteur économique est entièrement séparé du rendu (section 21 du cahier des charges) : aucune fonction de `src/engine/` ne touche au DOM, ce qui permet de le simuler et de le tester à grande vitesse (des runs de plusieurs jours simulés s'exécutent en quelques secondes).

Module	Rôle
<code>src/engine/mapDefinitions.js</code>	Définitions data-driven des maps (MapDefinition) : phases du cycle, familles d'amélioration, prix, seuils de progression.
<code>src/engine/mapEconomy.js</code>	Moteur pur production→stockage→transport : tick, calcul hors-ligne par formule, détection de goulot, phase d'animation courante.
<code>src/engine/mapUpgrades.js</code>	Achats d'améliorations de chaîne, génériques aux 8 familles.
<code>src/engine/meta.js</code>	Perles, atouts permanents, Renaissance (prestige).
<code>src/engine/state.js</code>	État initial d'une partie et d'une map.
<code>src/engine/simulation.js</code>	Orchestration : tick/offline sur toutes les maps possédées, modificateurs de méta-progression, déblocage de map suivante.
<code>src/engine/migrations.js</code>	Migration de sauvegarde V0→V1 et filet de sécurité champs manquants.
<code>src/ui/scene.js</code>	SVG du puits/stockage/citerne/travailleur, animation pilotée par la phase du moteur.
<code>src/ui/hud.js</code>	En-tête, tiroir à onglets (chaîne / atouts).
<code>src/ui/onboarding.js</code>	Tutoriel contextuel, une fois par mécanique, jamais réinitialisé.
<code>src/ui/prestige.js</code>	Écran de Renaissance : perdu/conservé/gagné explicite.
<code>src/main.js</code>	Assemblage, boucle principale, cycle de vie mobile.

3. Cycle exact du travailleur (Map 1 — l'eau)

8 phases, dans l'ordre, jamais de téléportation entre postes :

Phase	Durée de base	Famille qui la réduit
<code>prepare</code>	500 ms	fixe
<code>walkToWell</code>	1 800 ms	Déplacement
<code>lowerBucket</code>	1 200 ms	Corde / treuil
<code>wellFill</code>	2 400 ms	Puits
<code>raiseBucket</code>	1 200 ms	Corde / treuil
<code>walkToStorage</code>	1 800 ms	Déplacement
<code>pour</code>	500 ms	fixe
<code>walkBack</code>	1 200 ms	Déplacement

Cycle de base (aucune amélioration) : 10,6 s pour 1 L. Entièrement amélioré (niveau 4 sur seau/déplacement/treuil/puits) : ≈4,3 s pour 6 L, soit un débit environ 14,7x supérieur — chaque famille d'amélioration a un effet mesurable et distinct (testé explicitement, voir section 9).

L'UI lit la phase en cours via `currentProducerPhase()` (position d'animation interpolée, jamais un saut) ; quand le stockage est plein, le travailleur reste visuellement « en attente » (bulle d'exclamation, seau plein) au lieu de disparaître ou de bloquer le jeu.

4. Chaîne production → stockage → transport

Le débit effectif de la chaîne est $\min(\text{débit_production}, \text{débit_transport})$. Le maillon le plus lent est identifié en continu (`bottleneckKind`) et affiché en toutes lettres à l'écran, sans écran de statistiques à ouvrir : « La production limite le débit » ou « Le transport limite le débit » ou « Stockage plein ».

Vérifié par simulation (section 9) : surinvestir la production sans toucher au transport produit bien une saturation observable (96 L déjà lâchés avant blocage), et améliorer ensuite le transport (capacité + fréquence) débloque la situation — le déséquilibre est créé par le joueur, jamais un game over, et reste récupérable.

5. Liste des améliorations et effets

Famille	Effet économique	Effet visuel
Seau	+ litres par cycle	Seau visuellement plus grand, plein pendant le transport
Déplacement	Marche plus courte (walkToWell/Storage/Back)	Le travailleur marche plus vite à l'écran
Corde / treuil	Lower/raise plus courts	Remontée du seau visiblement accélérée
Puits	wellFill plus court	Le puits produit plus vite
Stockage	+ capacité de tampon	Le niveau visible dans la citerne monte plus haut avant saturation
Citerne	+ capacité chargée par passage	Camion visuellement plus grand
Fréquence transport	Intervalle plus court entre deux passages	Le camion revient plus souvent
Productivité	Multiplicateur d'argent par litre livré	— (reste volontairement numérique, section 7)

8 familles × 2 à 4 niveaux chacune (le premier niveau est le départ gratuit) — chaque achat change à la fois un nombre et quelque chose de visible, comme demandé en section 7 ; aucune n'est un simple « +10 % » sans conséquence perceptible.

6. Onboarding et persistance des tutoriels

7 messages contextuels (src/ui/onboarding.js), chacun déclenché une seule fois sur toute la vie de la sauvegarde et jamais réinitialisé par une Renaissance (state.tutorial.seen, en dehors de tout ce que le prestige remet à zéro) : première amélioration, première amélioration du puits, première amélioration du travailleur, première saturation, première amélioration de transport, première collecte hors-ligne, premier déblocage de map. Un écran « Aide » (bouton « ? ») permet de les revoir à tout moment.

Simplification assumée : l'explication apparaît au bon moment mais ne verrouille pas physiquement le reste de l'interface (pas d'overlay bloquant forcé) — un « agir pour comprendre » sans verrou dur, plus proche de l'esprit que de la lettre de la section 8.

7. Progression hors-ligne : formule, plafond, protections

Calcul par formule (jamais une simulation seconde par seconde d'une absence de 24h, comme explicitement demandé) : en régime permanent, le débit réel est borné par le maillon le plus lent — $\text{effectiveRate} = \min(\text{débit_production}, \text{débit_transport}) \times \text{productivité} \times \text{efficacité_hors_ligne}$. Le tampon est reconstruit de façon honnête et approximative après coup : plein et travailleur en pause si le transport était le goulot, vide sinon.

- Plafond par défaut : 4h, à 80% d'efficacité — extensible via l'atout « Cap hors-ligne » (+2/+5/+10h cumulés) et « Efficacité hors-ligne » (+10%/+20%, plafonné à 100%).
- Crédité une seule fois par retour réel : testé explicitement (idempotence face à des signaux de reprise redondants, voir section 1.3).

- Garde-fous contre les deltas absurdes : une absence négative ou nulle ne rapporte jamais rien ; un delta énorme (horloge système faussée) reste fini et positif, jamais NaN.
- Résumé honnête affiché au retour : durée réelle, revenu réel, mention explicite du plafond atteint le cas échéant.

Absence	Litres livrés	Argent gagné
15.0 min	84.9 L	169.8
1.0 h	339.6 L	679.2
4.0 h	1358.5 L	2717.0
8.0 h	2717.0 L	5434.0
1.0 j	8150.9 L	16301.9

Mesuré au niveau 1 partout (aucune amélioration) — une chaîne développée livre proportionnellement plus.

8. Maps : structure et contenu réellement livré

Architecture data-driven prête (mapDefinitions.js, MAP_ORDER) : ajouter une map suivante ne demande pas de dupliquer le moteur, seulement une nouvelle entrée de définition. **Une seule map est réellement autorée dans cette V1 : « La chaîne de l'eau ».** Aucune Map 2 n'a été construite — section 25 autorise explicitement à livrer l'architecture sans contenu supplémentaire si le temps ne permet pas de le faire proprement ; le construire à la hâte aurait risqué de dupliquer du moteur au lieu de le généraliser.

Le déblocage de map suivante (seuil à 50-75% d'utilité, section 11.1) est implémenté (checkMapUnlocks) et testé pour ne jamais planter en l'absence de map suivante définie — mais ne se déclenche jamais concrètement puisqu'aucune Map 2 n'existe encore. « Les anciennes maps continuent en arrière-plan » (section 12) est également implémenté de façon générique (applyTick fait avancer toutes les maps possédées, pas seulement celle affichée) mais n'a, avec une seule map, rien à démontrer concrètement pour l'instant.

9. Perles, Renaissance, atouts : règles réellement implémentées

9.1 Perles et Renaissance

perlesEarnable = floor(sqrt(argent_gagné_cette_run) × bonus_atout) — les premiers gains sont rapides, la suite ralentit sans jamais s'arrêter. Seuil minimal fixé à 100 Perles (hypothèse de calibrage explicite du cahier des charges, section 14.1, non figée : RENAISSANCE_MIN_PERLES est une constante isolée, changeable sans toucher au reste).

Chaque Renaissance accordée ajoute un bonus permanent cumulatif de +10% de production (PRESTIGE_PRODUCTION_BONUS_PER_RUN, également une hypothèse de calibrage isolée) : mesuré par simulation, une deuxième run atteint la Map 1 complète 1.35x plus vite que la run de référence sans jamais renaître (1.0 j contre 18.4 h) — l'accélération est réelle, mesurée, mais reste modeste avec un seul cran de bonus ; elle s'accumule à chaque Renaissance ultérieure.

Écran de confirmation (src/ui/prestige.js) : liste explicitement Perles à gagner, bonus permanent obtenu, ce qui sera réinitialisé (progression de la map, argent) et ce qui sera conservé (Perles, atouts, tutoriels vus, réglages audio) — deux boutons distincts, « Renaître maintenant » et « Continuer cette run », jamais de reset accidentel.

9.2 Atouts permanents (8 familles)

Atout	Effet	Coûts (Perles)
productionGlobal	+X% de production sur toute chaîne	5 / 15 / 40
cycleSpeed	-X% de durée de cycle	8 / 25 / 60
offlineEfficiency	+X% d'efficacité hors-ligne (plafonné à 100%)	10 / 30
storageGlobal	+X% de capacité de tampon	8 / 25
logistics	-X% d'intervalle de transport	12 / 35
upgradeCostReduction	-X% sur le coût de toute amélioration de chaîne	15 / 45
offlineCap	+X heures de plafond hors-ligne	10 / 30 / 70

perleGain	+X% de Perles gagnées à chaque Renaissance	20 / 60
-----------	--	---------

Génériques par construction : aucun ne référence un objet précis d'une map (pas de « seau +10% »), donc valables sur la future map agricole/forestière/autre sans modification.

10. Simulations économiques et hypothèses (section 20)

Profil	Map 1 complète	dont temps ACTIF	Renaissance dispo.
Léger : 2 sessions/jour de 1 min	3.5 j	7.0 min	1.0 j
Modéré : 3 sessions/jour de 3 min	2.0 j	18.0 min	16.0 h
Assidu : 5 sessions/jour de 5 min	1.2 j	30.0 min	9.6 h

Constat honnête, pas maquillé : l'hypothèse « Map 1 $\approx$ 1-2h de jeu ACTIF cumulé » (section 11) n'est pas confirmée telle quelle — le temps actif mesuré va de 7 à 30 minutes selon le profil, bien en dessous de l'hypothèse. La cause identifiée par la simulation elle-même : la progression hors-ligne plafonnée (jusqu'à 4h à 80% d'efficacité par absence) fait l'essentiel du travail entre les sessions, puisque le débit de base (sans aucune amélioration) est déjà d'environ 340 L/h. Un premier passage à des seuils 20x plus bas (3 000 / 6 000 L) finissait même la map en quelques minutes de jeu actif réel, quel que soit le profil — détecté immédiatement par cette même simulation avant tout test humain, exactement le rôle qu'on lui demandait de jouer. Les seuils ont été élargis (60 000 / 120 000 L) en conséquence ; la tension structurelle entre « l'offline est un pilier de rétention » et « la boucle active doit occuper 1-2h » reste réelle et est reportée en recommandation V2 plutôt que masquée.

- Goulot d'étranglement créé par le joueur : saturation observée, récupérable après amélioration du transport — confirmé (section 4).
- Prestige : premier seuil atteint après 1.5 h de jeu actif continu — largement avant la fin d'une run complète, cohérent avec « avant la lassitude » sans être précoce au point d'être trivial.
- Accélération post-prestige : x1.35 mesuré, réel mais modeste avec un seul cran de bonus.
- Atouts : 8 familles à coûts de premier niveau distincts (5 à 20 Perles), aucun strictement dominant sur les autres.
- Maps suivantes : question sans objet, une seule map autorée (voir section 8).

Reproductible : `npm run simulate` (V0, conservé) et un nouveau node scripts/simulate-map1.mjs pour cette V1, résultats écrits dans docs/haven-v1-simulation-results.json.

11. Tests exécutés et résultats exacts

37 tests node:test (moteur, aucune dépendance externe) + 3 vérifications Playwright (mobile), tous verts au moment de la livraison.

Domaine (section 28)	Couverture
Économie	production, coûts, achats (refus verrouillé/trop cher/max), goulots, saturation, revenus — tests/economy.test.js, tests/upgrades.test.js
Hors-ligne	15 min/1h/4h/8h/24h, plafond, efficacité réduite, absence négative/nulle, horloge anormale, idempotence face à des signaux redondants — tests/economy.test.js + check-mobile-resume.mjs

Sauvegarde	reload, fermeture, migration V0→V1, champs absents/partiels, conservation exacte au prestige — tests/save.test.js, tests/migrations.test.js, tests/meta.test.js
Mobile	clic après suspend/resume simulé, absence de débordement à 5 largeurs, aucune erreur console — scripts/check-mobile-resume.mjs (CI)
Animation	phases dans l'ordre sans saut, pause visible en saturation, cycles longs répétés sans dérive numérique — tests/economy.test.js
Maps	dépassement de seuil sans map suivante définie : ne plante jamais, documenté explicitement — tests/simulation.test.js
Prestige	seuil minimal, reset exact de la progression de run, conservation exacte de Perles/atouts/tutoriels/audio — tests/meta.test.js
Atouts	achat, refus au niveau maximum, réduction de coût réellement appliquée au débit — tests/meta.test.js, tests/upgrades.test.js
Tutoriel	persistance après Renaissance — tests/meta.test.js
Build	build de production propre, aucune erreur console dans tous les scénarios testés en navigateur
Déploiement	workflow build ET deploy verts sur les 4 derniers push (section 13) ; URL non re-fetchée depuis cette session (voir limite ci-dessous)

12. Performances et limites connues

- Une seule map réellement jouable (« La chaîne de l'eau ») — voir section 8.
- Onboarding sans verrou dur (simplification assumée, section 6).
- Tension active/hors-ligne non résolue dans cette V1 (section 10) : le temps actif mesuré reste sous l'hypothèse du cahier des charges.
- Diagnostic mobile validé par audit + test automatisé, pas encore par un appareil iOS réel (section 1.3).
- Art de la scène volontairement minimal (formes géométriques simples) — priorité donnée à la lisibilité du cycle et à la performance, pas au rendu.
- Aucune activité en dehors de la Map 1 : les « anciennes maps continuent en arrière-plan » est générique et fonctionnel, mais rien à observer concrètement sans Map 2.
- Modèle commercial non intégré (volontairement, décision de prudence du cahier des charges) : aucune interface de rewarded/premium préparée dans cette passe faute de temps restant — reporté en V2.

Performance : animations limitées à un seul travailleur, un camion, un remplissage de tampon — pas de création/destruction DOM par cycle (le SVG est statique, seuls attributs et classes changent). Aucune dégradation observée en session de test prolongée.

13. État du build et du déploiement

4 push sur main pendant cette mission, chacun avec le workflow « **Deploy to GitHub Pages** » (build + tests + test:mobile + déploiement) vert de bout en bout sur le runner GitHub Actions réel — pas seulement en local, y compris le job deploy-pages lui-même. URL stable inchangée :

<https://scuillerjack-oss.github.io/haven-v0/>.

Limite honnête : cette session tourne dans un environnement dont la politique réseau bloque les domaines externes non listés, y compris github.io — je n'ai donc pas pu re-charger moi-même l'URL publiée pour la vérifier visuellement depuis cette session, seulement confirmer via l'API GitHub que le job de déploiement a réussi. Un chargement réel de l'URL sur un téléphone reste la vérification qui compte, et n'a pas encore été refaite pour cette V1 précise.

14. Éléments volontairement reportés

- Map 2 (contenu jouable) — architecture prête, non autorée (section 8).
- Verrouillage dur de l'onboarding (action obligatoire bloquante) — simplifié en indice non bloquant.
- Interfaces de préparation commerciale (rewarded/premium) — non commencées faute de temps, direction déjà actée dans le cahier des charges.
- Confirmation humaine sur appareil réel du correctif mobile et de la première expérience/premier prestige (section 17, explicitement demandée comme non-substituable par la simulation).

15. Recommandations pour la V2 (basées sur cette bêta, pas des suppositions)

- Confronter le diagnostic mobile à un vrai test sur l'appareil où le bug a été observé, avant toute nouvelle fonctionnalité.
- Résoudre la tension active/hors-ligne mesurée en section 10 : soit réduire le plafond hors-ligne par défaut, soit indexer le déblocage de map/prestige sur le nombre d'achats effectués plutôt que sur les seuls litres livrés, pour que la boucle active reste le moteur principal même avec un hors-ligne généreux.
- Construire une Map 2 réelle sur l'architecture déjà prête, en réutilisant intégralement mapEconomy.js/mapUpgrades.js sans dupliquer de logique.
- Verrouiller réellement la première occurrence de chaque tutoriel (action obligatoire) si un test humain montre que le joueur les ignore trop souvent en l'état actuel (indice non bloquant).
- Étudier les interfaces rewarded/premium seulement après un test de rétention humain réel sur cette V1 — pas avant, comme le cahier des charges le demande explicitement.

Fin du rapport — HAVEN V1, dépôt scuillerjack-oss/haven-v0, branche main.